Anomaly Detection with Autoencoders

MS Data Engineering — Module 2, Blocks 1-2

Sarra Lajnef
Based on: Course GLT-polito

May 4, 2026

Contents

1 IEEE-CIS Fraud Detection **3**

2 Fraud Detection Train-Validation-Test Splits **3**
2.1 Splitting Ratios . . . 3

3 RAM Usage Optimization **3**

4 Baseline Model: Isolation Forest **4**
4.1 Experimental Results . . . 4
4.1.1 Test Set Classification Report . . . 4

5 Analysis of Isolation Forest Results **4**
5.1 High-Level Metrics: AUC and F1 . . . 5
5.2 Classification Deep Dive (Class 1: Fraud) . . . 5
5.3 The Validation Threshold . . . 5

6 Fraud Detection: AutoEncoder for Mixed Variables **5**
6.1 Categorical Variables and Embeddings . . . 5
6.2 Preprocessing and Indexing Strategy . . . 6
6.2.1 Indexing Example . . . 6

7 Loss Function and Optimization **6**
7.1 Categorical Reconstruction and Total Loss . . . 6

8 Training and Model Selection **7**
8.1 Preprocessing Pipeline . . . 7
8.2 Hyperparameter Validation Strategy . . . 7

9 Training Workflow and Final Evaluation **7**
9.1 The Learning Process and Negative Loss . . . 7
9.2 The Final Report Card . . . 7
9.3 Precision vs. Recall: Fraud Detection (Class 1) . . . 8
9.4 Conclusion: Why the Autoencoder Succeeded . . . 8

10 The Impact of Noise: Denoising Autoencoders **8**
10.1 Prevention of Data Memorization . . . 8
10.2 Robust Representation: Shapes vs. Points . . . 8
10.3 Enhanced Sensitivity and Recall Boost . . . 8

11 Fraud Detection Exercise: Tribute to Michael Jahrer (Part I) 9
11.1 Conceptual Overview: The Quantile Transform . . . 9
11.1.1 Mapping Logic . . . 9
11.2 Exercise Requirements . . . 9

12 Fraud Detection - Exercise: Tribute to Michael Jahrer (Part II) 10
12.1 Introducing Noise in Categorical Variables . . . 10
12.2 Implementation Objective . . . 10
12.3 Key Information . . . 10
12.4 Tasks . . . 10

13 ECG Analysis and One-Class SVM Pipeline (Sarra) 10
13.1 Visual Interpretation of Signals . . . 10
13.1.1 The "Normal" Row (Top) . . . 10
13.1.2 The "Anomalous" Row (Bottom) . . . 11
13.2 Pipeline Description (Comments by Sarra) . . . 11

14 One-Class SVM Results Analysis (Sarra) 11
14.1 1. The Good: Exceptional “Catch” Rate (Recall) . . . 11
14.2 2. The Trade-off: High False Alarm Rate (Precision) . . . 11
14.3 3. The “Generalization” Check . . . 11

15 Comparison: One-Class SVM vs Isolation Forest (Sarra) 12
15.1 Head-to-Head Comparison . . . 12
15.1.1 1. The “Safety” Winner: One-Class SVM . . . 12
15.1.2 2. The “Efficiency” Winner: Isolation Forest . . . 12
15.1.3 3. The “Reliability” Metric: AUC . . . 12
15.2 Summary: Which one is “Perfect”? . . . 12

16 LSTM Model and Optimizers for ECG Anomaly Detection 12
16.1 Model Overview . . . 12
16.2 Role of the Optimizer . . . 13
16.3 1. Adam (Adaptive Moment Estimation) . . . 13
16.4 2. RMSprop (Root Mean Square Propagation) . . . 13
16.5 Summary Insight . . . 13

1 IEEE-CIS Fraud Detection

We use a preprocessed version of the **IEEE-CIS Fraud Detection** dataset, originally released for the Kaggle competition: IEEE-CIS Fraud Detection. It is a large real-world e-commerce transaction dataset for fraud detection, combining transaction-level information with identity-related features.

1. We adopted the preprocessing and feature engineering strategy described in the notebook: EDA and Models by artgor.
2. We then applied an additional preprocessing step to make the dataset manageable in Colab: we retained **100,000 rows** and only the **top 50 features** according to feature importance.

Although the dataset was originally proposed for a **supervised classification** task, in this project we treat it as an **anomaly detection** problem in order to test our methods.

This dataset presents several important challenges:

- A large amount of **missing data**.
- **Continuous variables** that are far from being normally distributed.
- **Categorical variables** with high cardinality (many distinct categories).

2 Fraud Detection Train-Validation-Test Splits

We split the dataset by separating **normal transactions** (`isFraud = 0`) from **fraudulent transactions** (`isFraud = 1`).

The **training set** is built using **only normal samples**. In this way, the anomaly detection models learn the structure of regular behavior and are not exposed to fraud examples during training.

2.1 Splitting Ratios

The normal data are split into:

- **70%** for training
- **12.5%** for validation
- **12.5%** for test

The fraud data are **never used in training**. They are split only between:

- **50%** for validation
- **50%** for test

3 RAM Usage Optimization

To manage the high memory demands of the IEEE-CIS dataset within the limitations of the environment, the following strategies were implemented:

- **Disk Persistence:** All processed arrays were saved to disk. This ensures that data can be reloaded in its exact processed state and allows for in-place modifications, reducing the need for duplicate objects in memory.

- **Manual Memory Management:** Intermediate objects were explicitly removed using the `del` command.
- **Garbage Collection:** While `del` flags objects for removal, it does not always trigger an immediate release of physical RAM. To force the system to reclaim memory immediately, we explicitly invoked the Python Garbage Collector using `gc.collect()`.

4 Baseline Model: Isolation Forest

The Isolation Forest algorithm operates by assigning a "suspicion score" to each transaction based on its isolation depth; transactions that are easier to isolate are deemed more anomalous.

To convert these continuous scores into binary classifications (Fraud vs. Normal), we performed a threshold optimization. We evaluated 200 potential cutoff points against the validation set to maximize the **F1-score**. This metric was chosen because it provides a balanced assessment, penalizing both missed fraud (False Negatives) and the incorrect flagging of legitimate users (False Positives).

Once the optimal threshold was identified on the validation data, it was locked and applied to the final test set.

4.1 Experimental Results

The table below summarizes the performance of the Isolation Forest baseline using the optimized threshold.

Metric	**Value**
Best Validation Threshold	-0.0425
Best Validation F1-score	0.3461
Validation AUC	0.7088
Test F1-score	**0.3567**
Test AUC	**0.7121**

Table 1: Isolation Forest Performance Summary

4.1.1 Test Set Classification Report

```
              precision    recall  f1-score   support

           0     0.9315    0.7094    0.8054     12057
           1     0.2465    0.6456    0.3567      1775

    accuracy                         0.7012     13832
   macro avg     0.5890    0.6775    0.5811     13832
weighted avg     0.8436    0.7012    0.7478     13832
```

5 Analysis of Isolation Forest Results

The baseline Isolation Forest model provides a foundational performance benchmark. Below is a detailed breakdown of the high-level metrics and the classification performance.

5.1 High-Level Metrics: AUC and F1

- **Test AUC (0.7121):** This indicates a 71.2% probability that the model will assign a higher fraud score to a randomly selected fraudulent transaction than to a random normal one. While this is significantly better than a random baseline (0.50), it leaves substantial room for improvement via more complex architectures, such as neural networks.
- **Test F1-score (0.3567):** As the harmonic mean of Precision and Recall, a score of approximately 0.36 is relatively low. This suggests that the model currently struggles to achieve a balance between capturing fraud and maintaining accuracy.

5.2 Classification Deep Dive (Class 1: Fraud)

Analyzing the specific performance for the fraudulent class reveals the model's strengths and weaknesses:

- **Precision (0.2465):** This serves as our "False Alarm" metric. When the model flags a transaction as fraud, it is correct only 24.6% of the time. Consequently, roughly 75% of the flagged transactions are actually legitimate (False Positives).
- **Recall (0.6456):** This represents the "Catch Rate." The model successfully identified 64.5% of all fraud cases in the test set but failed to detect approximately 35% of fraudulent activity (False Negatives).
- **Support (1775):** This denotes the total number of fraudulent transactions present in the test set.

5.3 The Validation Threshold

The Isolation Forest algorithm outputs a raw "anomaly score." To convert this into a binary decision, we utilized a **Best Validation Threshold of -0.0425**.

By iterating through 200 possible cutoff points, this specific value was found to maximize the F1-score on the validation data. In the final implementation, any transaction with a raw score higher than this threshold is labeled as **Fraud**.

6 Fraud Detection: AutoEncoder for Mixed Variables

The proposed AutoEncoder architecture is specifically designed to handle **mixed tabular data**, incorporating both **continuous** and **categorical** variables within a single neural framework.

6.1 Categorical Variables and Embeddings

Rather than utilizing traditional one-hot encoding—which can be computationally expensive and memory-intensive for high-cardinality features—this model employs **Embedding Layers** for categorical data:

- Each categorical feature is passed through its own dedicated embedding layer.
- These layers map discrete category indices into dense, low-dimensional learned vectors.
- This approach is significantly more **memory-efficient** and allows the model to learn latent relationships between different categories.

6.2 Preprocessing and Indexing Strategy

To ensure the model is robust to real-world data, each categorical column is re-indexed based on the **training set only**.

1. **Training Phase:** Observed categories are assigned consecutive integers starting from zero.
2. **Validation and Testing:** Any category appearing in the validation or test sets that was not present during training is mapped to a dedicated **unknown token** (max_index + 1).

6.2.1 Indexing Example

Consider a categorical feature where the transformer is fit on the training data:

- **Training input:** `X_train_cat[:, cat_id] = [15, 27, 2, 15, 3]`
- **Transformed output:** `[0, 1, 2, 0, 3]`

If unseen categories appear during inference:

- **Test input:** `X_test_cat[:, cat_id] = [27, 12, 15, 33]`
- **Transformed output:** `[1, 4, 0, 4]`

*Note: In this example, 4 serves as the **unknown token** for values 12 and 33, which were not seen during training.*

---

7 Loss Function and Optimization

For the **continuous variables**, the model deviates from standard Mean Squared Error (MSE) by interpreting the reconstruction term as the **negative log-likelihood of a Gaussian distribution**. For each continuous feature, the decoder predicts a dual output:

- A **mean** (μ)
- A **standard deviation** (σ) or variance parameter

This approach normalizes the anomaly score relative to the **intrinsic variability** of each feature. Consequently, reconstruction errors on naturally noisy variables are penalized less severely than errors of the same magnitude on stable, low-variance variables.

7.1 Categorical Reconstruction and Total Loss

Reconstruction for categorical variables is managed via a **cross-entropy loss** over predicted category probabilities. This is mathematically equivalent to the **negative log-likelihood of a categorical distribution**.

The total loss (L) is defined as a weighted combination of the continuous and categorical components:

$$L = p\, L_{cont} + (1 - p)\, L_{cat}, \quad p \in [0, 1] \tag{1}$$

---

8 Training and Model Selection

8.1 Preprocessing Pipeline

Before training, continuous variables undergo a two-step preprocessing sequence:

1. **Imputation:** Missing values are filled using the feature mean.
2. **Scaling:** A `StandardScaler` is applied to ensure zero mean and unit variance.

8.2 Hyperparameter Validation Strategy

The primary metric for model selection is the **Area Under the Curve (AUC)** calculated on the validation set. Our optimization strategy prioritized hyperparameters in the following order:

1. **Loss Weights:** Adjusting the balance (p) between continuous and categorical reconstruction.
2. **Architecture vs. Regularization:** Balancing the **latent embedding size** against the intensity of **Gaussian noise injected** into continuous variables during training to prevent overfitting.

   Note: This framework establishes a complete "Neural Network laboratory" for fraud detection, capable of processing multi-modal tabular data to identify complex anomalous patterns.

9 Training Workflow and Final Evaluation

The training process followed a structured "laboratory" pipeline:

1. **Data Pipeline Construction:** Building the loaders (`build_dataloaders`).
2. **Hyperparameter Tuning:** Executing a grid search to test different "recipes" (`grid_search`).
3. **Model Optimization:** Training via data compression and reconstruction (`train_one_config`).
4. **Anomaly Scoring:** Quantifying the "surprise" level of each transaction (`anomaly_scores`).
5. **Performance Reporting:** Calculating final metrics and accuracy (`evaluate_scores`).

9.1 The Learning Process and Negative Loss

During the 20 training epochs, we observe a steady decline in `loss` and `cont_nll` (continuous negative log-likelihood).

A notable observation is the **negative loss values**. This is not a mathematical error; because we use Log-Likelihood, a negative value indicates the model is becoming increasingly "certain" and precise in its definition of a normal transaction. Simultaneously, the dropping `cat_ce` (Categorical Cross-Entropy) shows the model is successfully learning the card types and categories typical of legitimate users.

9.2 The Final Report Card

The evaluation of the "Champion Model" yielded the following performance metrics:

- **Overall Accuracy (0.8200):** The model correctly classifies transactions 82% of the time.
- **Weighted Average (0.8305):** This score accounts for the class imbalance (the prevalence of normal transactions over fraud). A score of 0.83 is considered very strong for this type of unsupervised anomaly detection.

9.3 Precision vs. Recall: Fraud Detection (Class 1)

Focusing specifically on the model's ability to identify fraudulent activity:

- **Recall (0.6681):** The Autoencoder successfully captures approximately **67%** of all fraud cases. This represents a measurable improvement over the **64%** recall achieved by the baseline Isolation Forest.
- **Precision:** By reaching 82% accuracy, the model has significantly reduced "False Alarms." It is more confident in its detections, meaning a flagged transaction is more likely to be actual fraud compared to the baseline.

9.4 Conclusion: Why the Autoencoder Succeeded

The improvement is largely attributed to the balanced `cont_weight` (set at 0.5 or 0.75). Unlike the Isolation Forest, which may be "distracted" by isolated numerical outliers, the Autoencoder learns the complex **latent relationships** between numerical amounts and categorical identities, leading to a more holistic understanding of transaction behavior.

10 The Impact of Noise: Denoising Autoencoders

The introduction of Gaussian noise into the continuous variables transforms the model into a **Denoising Autoencoder**. This strategic "blurring" of the input data provides three primary advantages:

10.1 Prevention of Data Memorization

Standard Autoencoders are prone to "cheating" by simply learning an identity function—essentially memorizing training samples and copying inputs directly to outputs. When noise is added, the model can no longer rely on a simple "mirror" effect. It is forced to learn the underlying structure of the data to reconstruct the original, clean input from the corrupted version.

10.2 Robust Representation: Shapes vs. Points

The inclusion of noise shifts the model's learning objective from specific data points to generalized distributions:

- **Without Noise:** The model might learn that a specific transaction (e.g., $10.50 at a grocery store) is normal, creating an overly rigid definition.
- **With Noise:** The model learns that a *range* of values around $10.50 is acceptable.

This creates a smoother and more robust decision boundary, leading to a more realistic definition of "normal behavior."

10.3 Enhanced Sensitivity and Recall Boost

The transition to a Denoising model is the primary driver for the observed increase in **Recall** (70% compared to the initial 64%).

Fraudulent transactions are often designed to mimic legitimate behavior, differing only by subtle deviations. Because the Denoising model understands the "essence" of a normal transaction rather than just its surface-level values, it is significantly harder to "fool." It becomes sensitive to tiny, clever anomalies that a standard model would likely dismiss as being "close enough" to the training data.

11 Fraud Detection Exercise: Tribute to Michael Jahrer (Part I)

In this exercise, we introduce a powerful technique for data normalization: the **Quantile Transform**. This method is often considered a critical component in high-performance tabular data models.

11.1 Conceptual Overview: The Quantile Transform

The objective of the Quantile Transform is to reshape the empirical distribution of a variable so that it approximates a **Gaussian** or **Uniform** distribution. Unlike the `StandardScaler`, which only adjusts for mean and variance, the Quantile Transform reshapes the entire distribution.

- **Fit Phase:** The `fit(X_train)` function computes the empirical quantiles of each continuous feature using the **training set only**.
- **Transform Phase:** The `transform(X)` function maps each value to its relative position in the learned cumulative distribution. It then converts that position to the corresponding value of the target distribution.

11.1.1 Mapping Logic

- Values in the **lower tail** are mapped to large negative values (Normal) or values near 0 (Uniform).
- Values in the **center** are mapped near 0 (Normal) or 0.5 (Uniform).
- Values in the **upper tail** are mapped to large positive values (Normal) or 1 (Uniform).

Note: For validation and test data, the transformer uses the mapping learned on the training set. Values outside the training range are mapped to the extreme tails of the transformed distribution.

11.2 Exercise Requirements

To complete this exercise, implement the following modifications to the pipeline:

1. **Preprocessing Update:** Replace the `StandardScaler` in the continuous variable pipeline with `QuantileTransformer(output_distribution="normal")` or `"uniform"`.
2. **Range Adjustment:** If using the **uniform** distribution, apply the linear transformation $y = 2x - 1$ to shift the range from $[0, 1]$ to $[-1, 1]$.
3. **Visualization:** Generate and plot histograms of the transformed continuous variables to verify the success of the distribution reshaping.
4. **Validation Scheme:** Repeat the hyperparameter optimization sequence:
   - Validate the **loss weights** (p vs $1 - p$).
   - Validate the **latent embedding size** against the level of **injected Gaussian noise**.
5. **Final Evaluation:** Assess the performance of the resulting "best model" on the test set.

---

12 Fraud Detection - Exercise: Tribute to Michael Jahrer (Part II)

12.1 Introducing Noise in Categorical Variables

How can we introduce noise in categorical variables?

A simple and effective idea is to **mask** a categorical value with probability `cat_noise`, by replacing it with the **unknown token** of that column. Recall that the unknown token is the maximum valid index for that categorical variable, and it is already used for categories that were not seen during training.

12.2 Implementation Objective

You have to implement this categorical corruption scheme.

Hint: follow exactly the same flow already used for `num_noise` in the code.

The new parameter `cat_noise` should:

- enter the training configuration in the same way,
- be passed through the same functions,
- and be applied in the same place in the training loop, that is, just before feeding the batch to the model.

12.3 Key Information

All the information you need is already contained in `embedding_sizes`:

- `embedding_sizes[i][0]` is the number of possible indices for categorical variable i
- therefore, the **unknown token** for variable i is:

$$\text{unknown token}_i = \texttt{embedding_sizes[i][0]} - 1$$

12.4 Tasks

You have to:

- validate `cat_noise` vs `cont_weight`, starting from the best model found in the previous step
- evaluate the best configuration on the test set

13 ECG Analysis and One-Class SVM Pipeline (Sarra)

13.1 Visual Interpretation of Signals

13.1.1 The "Normal" Row (Top)

Consistency: Notice that while they aren't identical, they all follow a similar pattern: a small bump, a sharp spike (the heartbeat), and a final wave.

Cleanliness: The lines are relatively smooth. The "Normal 1" and "Normal 3" plots show very classic, textbook heart signals.

13.1.2 The "Anomalous" Row (Bottom)

Chaos: Look at "Anomalous 3." It is extremely noisy and doesn't have that clear, single spike.

Deformation: Look at "Anomalous 4." The spike actually goes down (negative amplitude) very sharply compared to the normal ones.

Timing: The "spikes" in the anomalous row often happen at different times or have "double peaks" (like in Anomalous 1).

13.2 Pipeline Description (Comments by Sarra)

1. **FLATTENING:** Transform the $(160, 12)$ heartbeats into long 1D vectors (1920 features). The SVM cannot read "images" or "shapes"; it needs one long row of numbers.
2. **FILTERING:** Ensure the model ONLY sees normal beats ($y = 0$) during training.
3. **SCALING:** Use StandardScaler to give every time-step a mean of 0 and variance of 1.
4. **COMPRESSION (PCA):** Reduce the 1920 features down to the 50 most important "patterns."
   This removes noise and makes the math easier for the SVM.
5. **MODELING:** Initialize the One-Class SVM.
   $\nu = 0.05$ means we expect roughly 5% of our "normal" training data might be outliers.
6. **SCORING:** Calculate how far away each heartbeat is from the "Normal Boundary."
   We use a negative sign because `decision_function` returns positive for normal and negative for outliers.
7. **EVALUATION:** Measure the area under the ROC curve (AUC).
8. **THRESHOLD TUNING:** Test 200 different "sensitivity levels" to find the best F1-score.
9. **FINAL PREDICTION:** Use the "winning" threshold to make final Fraud/Normal guesses.

14 One-Class SVM Results Analysis (Sarra)

14.1 1. The Good: Exceptional "Catch" Rate (Recall)

Test Recall (0.9779): we caught 97.8% of the anomalous heartbeats. In a medical context, this is exactly what we want, the model rarely misses a problem.

Test AUC (0.9701): An AUC of 0.97 is nearly perfect. It means the model is incredibly good at ranking "Normal" vs. "Anomaly" correctly.

14.2 2. The Trade-off: High False Alarm Rate (Precision)

Test Precision (0.5412): This is where it's not "perfect". Only 54% of the beats we flagged as anomalies were actually anomalies.

Confusion Matrix: we had 2,475 False Positives (healthy beats flagged as anomalies). In a hospital, this would mean a lot of "beep-beep" alarms going off when the patient is actually fine.

14.3 3. The "Generalization" Check

Validation AUC (0.94) vs. Test AUC (0.97): Surprisingly, our model performed better on the Test set than the Validation set. This suggests our model is very robust and hasn't overfitted to the training data.

F1 Drop (0.92 to 0.69): The F1 score dropped because the Test set has a different proportion of anomalies than the Validation set. This is common in real-world data.

15 Comparison: One-Class SVM vs Isolation Forest (Sarra)

Comparing our One-Class SVM to this Isolation Forest, we can see a classic machine learning "trade-off." While our SVM was slightly better at catching every single anomaly, the Isolation Forest is actually a more "practical" model for a real-world hospital.

15.1 Head-to-Head Comparison

15.1.1 1. The "Safety" Winner: One-Class SVM

Recall (Catching Anomalies): The SVM caught 97.79% of anomalies, while the Isolation Forest caught 96.85%.

Why it matters: If your goal is "Zero Misses," the SVM is slightly safer. It missed only 66 beats, whereas the Isolation Forest missed 94.

15.1.2 2. The "Efficiency" Winner: Isolation Forest

Precision (False Alarms): The Isolation Forest is superior here (0.58 vs 0.54).

Confusion Matrix: The Isolation Forest had 2,084 false alarms, which is roughly 400 fewer than the SVM.

Test F1-Score: Because it balanced precision and recall better, the Isolation Forest has a higher Test F1 (0.726 vs 0.696).

15.1.3 3. The "Reliability" Metric: AUC

- Isolation Forest Test AUC: 0.9691
- One-Class SVM Test AUC: 0.9701

Verdict: These scores are nearly identical. Both models are doing an incredible job of ranking the signals from "normal" to "weird."

15.2 Summary: Which one is "Perfect"?

The Isolation Forest is likely the better choice here.

Even though it missed a few more anomalies (94 vs 66), it is much "quieter." In a medical setting, reducing false alarms by 400 while still maintaining a 96.8% recall is usually considered a major win. It saves the medical staff a lot of time without significantly compromising patient safety.

16 LSTM Model and Optimizers for ECG Anomaly Detection

16.1 Model Overview

The Model: Uses LSTMs to understand the "flow" of time in an ECG.

The Strategy: Only trains on Normal data so it gets very good at recognizing "healthy" patterns.

The Grid Search: Automatically tests different "sizes" of the hidden brain (`hidden_dim`) and different optimizers to find the best configuration.

The Evaluation: Uses ROC-AUC and F1-score to measure:

- how many anomalies are correctly detected (Recall)
- how many false alarms are produced (Precision)

16.2 Role of the Optimizer

In the context of the neural network, the **optimizer** is the "driver" of the model. It looks at the errors (loss) and decides how to adjust the internal weights to improve performance at each step.

16.3 1. Adam (Adaptive Moment Estimation)

Adam is the most popular optimizer today because it is fast and usually requires very little manual tuning. It combines two key ideas:

Momentum

Like a heavy ball rolling down a hill, it gains speed in directions that consistently lead downward. This helps it roll past small "bumps" or "puddles" (local minima) that might trap simpler optimizers.

Adaptive Learning Rate

It keeps track of how often each specific weight changes:

- If a weight is changing a lot, Adam slows down updates to be more precise
- If a weight is barely moving, Adam gives it a bigger "push"

16.4 2. RMSprop (Root Mean Square Propagation)

RMSprop was specifically designed to handle the **exploding** or **vanishing gradients** often found in RNNs and LSTMs.

The Problem

In sequential data (like ECG signals), gradient updates can become:

- extremely large (exploding gradients)
- extremely small (vanishing gradients)

which can break or stall the training process.

The Fix

RMSprop uses a moving average of recent squared gradients to **normalize** updates. It effectively acts as a "speed limiter," preventing weights from changing too fast and ensuring stable training.

16.5 Summary Insight

- Adam is generally faster and easier to tune
- RMSprop is particularly well-suited for sequential models like LSTMs
- Both optimizers aim to improve convergence, but handle learning dynamics differently